CPSC 250
Plant Information Example

Edward Brash

1 Program Goal

In this example, we use inheritance to build a small plant information program.

We are given:

- a base `Plant` class,
- a derived `Flower` class.

We want to create a list called:

```
my_garden
```

that stores both `Plant` and `Flower` objects.

2 The Base Plant Class

A plant has:

- a name,
- a cost.

```
class Plant:

    def __init__(self, plant_name, plant_cost):
        self.plant_name = plant_name
        self.plant_cost = plant_cost

    def print_info(self):
        print(f"Plant name: {self.plant_name}")
        print(f"Plant cost: {self.plant_cost}")
```

3 The Derived Flower Class

A flower is a plant, but it has additional information:

- whether it is annual,
- flower color.

```
class Flower(Plant):

    def __init__(self, plant_name, plant_cost,
                 is_annual, color_of_flowers):

        Plant.__init__(self, plant_name, plant_cost)

        self.is_annual = is_annual
        self.color_of_flowers = color_of_flowers

    def print_info(self):
        Plant.print_info(self)
        print(f"Annual: {self.is_annual}")
        print(f"Color of flowers: {self.color_of_flowers}")
```

4 The Input Format

The program repeatedly reads user input until the user enters:

```
-1
```

Each input line tells us whether the item is a plant or a flower.
Examples:

```
plant fern 12.99
flower rose 15.99 true red
-1
```

5 Processing the Input

We split the user input into tokens:

```
tokens = user_string.split()
```

Then:

```
type = tokens[0]
name = tokens[1]
cost = tokens[2]
```

If the type is `plant`, we create a `Plant` object.
If the type is `flower`, we create a `Flower` object.

6 Creating Objects

```
if type == "plant":

    my_plant = Plant(name, cost)
    my_garden.append(my_plant)

elif type == "flower":
```

```
    is_annual = tokens[3]
    color = tokens[4]

    my_flower = Flower(name, cost, is_annual, color)
    my_garden.append(my_flower)
```

7 The Main Loop

```
my_garden = []

user_string = input()

while user_string != "-1":

    tokens = user_string.split()

    type = tokens[0]
    name = tokens[1]
    cost = tokens[2]

    if type == "plant":

        my_plant = Plant(name, cost)
        my_garden.append(my_plant)

    elif type == "flower":

        is_annual = tokens[3]
        color = tokens[4]

        my_flower = Flower(name, cost, is_annual, color)
        my_garden.append(my_flower)

    user_string = input()
```

8 Printing the List

We write a function:

```
def print_list(my_garden):
```

that loops over every object in the list and calls its `print_info()` method.

```
def print_list(my_garden):

    for i in range(len(my_garden)):

        print(f"Plant {i + 1} Information:")
        my_garden[i].print_info()
        print()
```

9 Polymorphism

This example demonstrates polymorphism.

The list contains both:

- `Plant` objects,
- `Flower` objects.

But the loop simply calls:

```
my_garden[i].print_info()
```

Python automatically uses the correct version of `print_info()` depending on the actual object type.

10 Complete Program

```
class Plant:

    def __init__(self, plant_name, plant_cost):
        self.plant_name = plant_name
        self.plant_cost = plant_cost

    def print_info(self):
        print(f"Plant name: {self.plant_name}")
        print(f"Plant cost: {self.plant_cost}")


class Flower(Plant):

    def __init__(self, plant_name, plant_cost,
                 is_annual, color_of_flowers):

        Plant.__init__(self, plant_name, plant_cost)

        self.is_annual = is_annual
        self.color_of_flowers = color_of_flowers

    def print_info(self):
        Plant.print_info(self)
        print(f"Annual: {self.is_annual}")
        print(f"Color of flowers: {self.color_of_flowers}")


def print_list(my_garden):

    for i in range(len(my_garden)):
        print(f"Plant {i + 1} Information:")
        my_garden[i].print_info()
        print()


my_garden = []
```

```
user_string = input()

while user_string != "-1":

    tokens = user_string.split()

    type = tokens[0]
    name = tokens[1]
    cost = tokens[2]

    if type == "plant":
        my_garden.append(Plant(name, cost))

    elif type == "flower":
        is_annual = tokens[3]
        color = tokens[4]
        my_garden.append(Flower(name, cost, is_annual, color))

    user_string = input()

print_list(my_garden)
```

11 Key Takeaways

- A derived class can extend a base class.
- A derived class can override a method from the base class.
- Lists can store objects of related types.
- Polymorphism allows the same method call to behave differently depending on the object.
- Input parsing often begins by splitting strings into tokens.